PHASE 4a
Technical Document

UniGo



Developed by Saien Naidu
Grade 12
Information Technology
2025

Table of Contents

1. Externally Sourced Code 3
2. Explanation of Critical Algorithms 3
   2.1 reqMet() 3
   2.2 degreeFinder(Filter f) 4
3. Advanced Techniques 4
   3.1 Updating A Table 4
   3.2 Creating a Query with the Degree Finder 5

1. Externally Sourced Code

The Database Driver was sourced from elsewhere, and I only made slight modifications.

2. Explanation of Critical Algorithms

2.1 reqMet()

With all subjects, the user's mark has to be the same or higher than the mark specified.
For HL, FAL and MATHS, the user's subject choice has to be the same as the requirement's specification.

For the subject choices...it's different. I will use Physics as an example to explain the complexity
Let's say that Physics is a requirement of the Degree
Physics can be the user's 1st, 2nd or 3rd choice (opt1C, opt2C or opt3C)
And it can be in opt1C, opt2C or opt3C of the requirement
Therefore, a cross check has to be used:

The user's opt1C, opt2C and opt3C has to be compared with the degree's opt1C, opt2C and opt3C,
This gives us 5 possible combinations: 123, 132, 213, 231, 312 (321 will never happen)

If the user has Physics as their 1st choice, and the degree's opt1C is Physics, this will give us the combination: 123
If the user has Physics as their 2nd choice, and the degree's opt1C is Physics, this will give us the combination: 213
If the user has Physics as their 3rd choice, and the degree's opt1C is Physics, this will give us the combination: 231
If the user has Physics as their 1st choice, and the degree's opt2C is Physics, this will give us the combination: 213
If the user has Physics as their 2nd choice, and the degree's opt2C is Physics, this will give us the combination: 123
If the user has Physics as their 3rd choice, and the degree's opt2C is Physics, this will give us the combination: 132
If the user has Physics as their 1st choice, and the degree's opt3C is Physics, this will give us the combination: 312
If the user has Physics as their 2nd choice, and the degree's opt3C is Physics, this will give us the combination: 132
If the user has Physics as their 3rd choice, and the degree's opt3C is Physics, this will give us the combination: 123

The algorithm will try each combination once to determine if the user has chosen Physics.
If a combination works, then the user’s LO and APS will be checked
If neither of the 5 combinations work, the algorithm will move on to the next degree’s requirement

The user's LO mark has to be 50 or above

The user's APS score has to be the same or above the APS score specified

2.2 degreeFinder(Filter f)

When the filter is applied, this algorithm checks which degrees satisfies the filter and creates a list of them. The filter is layered out in different categories: Use user marks, location, faculty and university. If filters of different categories are applied, the degree must satisfy ALL the filters. If filters of the same category are applied, then the degree must satisfy at least one of the filters. For example, look at the following filter:

- Use user's marks
- KZN, Western Cape
- Commerce, Science
- UCT

The degree would have to satisfy the following:

(User meets req.) AND (KZN OR Western Cape) AND (Commerce OR Science) AND (UCT)
Or in other words:
The user has met the requirements, the degree is in KZN or Western Cape, the degree is in the Commerce or Science faculties, and the degree is offered by UCT

The algorithm will go through each category, eliminating more degrees (thus shortening the list) as the categories are checked. The algorithm will only check the degrees that satisfied the last category. For example:

For 'Use User Marks', the algorithm will check ALL the degrees in the DB. Let's say that 8 made it. Then, for the next category, the algorithm will only check those 8 degrees to determine if they satisfy the category. I.e. The algorithm does not check every degree, every category.

3. Advanced Techniques

3.1 Updating A Table

To create or change a table, the program needs a 2D array to act as the table's model. After determining the table's content, an algorithm is used to sort the data into a new 2D array. Since all the tables in the program only have one column, the first part of the 2D array does not need to be declared. This 2D array is then sent to the FrontEnd.

Additionally, the program also keeps track of the IDs of the objects in this 2D array (overwriting whatever is stored previously). If the table needs to be updated with the same set of objects, this public list of IDs can be called instead of the aforementioned algorithm.

3.2 Creating a Query with the Degree Finder

There are four different categories of filters that the user can apply, and three of them require a query to be sent to the database. In this example, I will be using the ‘Faculty’ category. For starters, we have the base query:

```
results = getDegWithQuery("""
                          SELECT
                              Degree_Table.ID,
                              DegreeName,
                              Degree_Table.UniversityID,
                              Degree_Table.FacultyID,
                              Degree_Table.Description
                          FROM
                              Degree_Table
                              INNER JOIN (
                                  University_Table
                                  INNER JOIN Faculty_Table ON University_Table.ID = Faculty_Table.UniversityID
                              ) ON (Degree_Table.UniversityID = University_Table.ID)
                              AND (Degree_Table.FacultyID = Faculty_Table.ID)
                          WHERE
                              Degree_Table.FacultyName LIKE """
        + faculties + ";");
```

The variable ‘faculties’ includes the list of the user’s selected faculties in a format that can be understood by MS Access. Using an algorithm, the program gets the faculties into said format. Only one aspect needs to be considered, and that is whether the current faculty (that the algorithm is adding to the query) is the first faculty in the query or not. If it is, simply add the faculty as is. If not, add the faculty along with whatever is necessary to adhere to the correct format. The full code can be seen below:

```
// Add a faculty to a query
private String addFaculty(String original, String toAdd) {
    // If there are currently no faculties in the query
    if (original.equals("")) {
        // Only return the faculty
        return toAdd;
    } else { // Otherwise
        // Return the new faculty appended to the others while maintaining the integrity of the query
        return original + "\nOR Degree_Table.FacultyName LIKE " + toAdd;
    }
}
```